МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

«Операции над мультимножествами и код Грея»

по дисциплине «Дискретная математика»

Выполнил:
студент гр. 5130201/XXXXX
И. О. Фамилия

Проверил:
преподаватель Востров А.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

Цель работы — изучение мультимножеств и операций над ними, а также алгоритма генерации бинарного кода Грея; программная реализация этих понятий на языке C++.

Задание. Реализовать программу генерации бинарного кода Грея для заполнения универсума мультимножеств заданной пользователем разрядности. На основе универсума формируются два мультимножества двумя способами заполнения: вручную и автоматически (способ выбирает пользователь). Мощности мультимножеств задаёт пользователь. На экран должны выводиться результаты операций над мультимножествами: объединение, пересечение, разность (оба варианта), симметрическая разность, дополнение (оба варианта), арифметические сумма, разность, произведение и деление. Кроме того, требуется реализовать защиту от некорректного пользовательского ввода и пользовательское меню.

Для достижения цели необходимо решить следующие задачи:

1) изучить определения мультимножества, универсума и операций над мультимножествами;

2) изучить алгоритм построения бинарного кода Грея и обосновать его корректность;

3) спроектировать структуру данных для хранения мультимножеств и разделить программу на модули;

4) реализовать генерацию универсума, ручное и автоматическое заполнение мультимножеств, все операции задания и консольное меню;

5) обеспечить защиту от некорректного ввода и протестировать программу, включая граничные случаи.

1 Математическое описание

1.1 Множества и мультимножества

Множество — это совокупность различных объектов, называемых его элементами. Если элементы совокупности могут повторяться, то такая совокупность называется мультимножеством [**?**]. Обычно все рассматриваемые множества являются подмножествами некоторого одного, достаточно широкого множества, которое называют универсальным множеством, или универсумом.

Пусть $X = \{x_1, \ldots, x_k\}$ — конечное множество (носитель). *Мультимножеством* A над X называется совокупность элементов X, в которую каждый элемент x_i входит $m_A(x_i) \in \mathbb{N}_0$ раз. Число $m_A(x)$ называется *кратностью* (показателем) элемента x в A; равенство $m_A(x) = 0$ означает, что x в A не входит. Мультимножество записывают в виде

$$A = \langle m_A(x_1)(x_1),\ \ldots,\ m_A(x_k)(x_k)\rangle.$$

Мощностью мультимножества называется сумма кратностей всех его элементов:

$$|A| = \sum_{x \in X} m_A(x). \tag{1}$$

В отличие от обычного множества, мощность мультимножества не совпадает с числом различных элементов в нём. Мультимножество, у которого все кратности равны нулю, называется пустым и обозначается $\varnothing$.

Универсумом U в работе называется мультимножество над тем же носителем X, кратности которого задают максимально допустимое число вхождений каждого элемента. Мультимножество A является подмультимножеством универсума ($A \subseteq U$), если

$$m_A(x) \le m_U(x) \quad \text{для всех } x \in X, \tag{2}$$

откуда следует $|A| \le |U|$.

В программе носителем X является множество всех кодов Грея заданной разрядности n, то есть $|X| = 2^n$, а кратности элементов универсума выбираются случайно из диапазона $[1; 7]$. Следовательно, $2^n \le |U| \le 7 \cdot 2^n$.

Пример. Для $n = 2$ пусть $U = \langle 4(00),\, 2(01),\, 5(11),\, 3(10)\rangle$ и $A = \langle 2(00),\, 0(01),\, 3(11),\, 1(10)\rangle$. Тогда $A \subseteq U$, $|U| = 14$, $|A| = 6$. В программе A выводится в виде , то есть элементы с нулевой кратностью не печатаются.

1.2 Операции над мультимножествами

Пусть A и B — мультимножества над носителем X, причём $A \subseteq U$ и $B \subseteq U$. Все операции определяются поэлементно: для каждого $x \in X$ кратность результата C вычисляется через значения $a = m_A(x)$, $b = m_B(x)$ и $u = m_U(x)$. Формулы, реализованные в программе, приведены в таблице **??**.

Таблица 1 – Операции над мультимножествами (здесь $a = m_A(x)$, $b = m_B(x)$, $u = m_U(x)$)

Операция	Обозначение	Кратность $m_C(x)$ в результате
Объединение	$A \cup B$	$\max(a,\, b)$
Пересечение	$A \cap B$	$\min(a,\, b)$
Дополнение	$\overline{A}$	$u - a$
Разность	$A \setminus B = A \cap \overline{B}$	$\min(a,\, u - b)$
Симметрическая разность	$A \triangle B = (A \cup B) \setminus (A \cap B)$	$\min\big(\max(a,b),\, u - \min(a,b)\big)$
Арифметическая сумма	$A + B$	$\min(a + b,\, u)$
Арифметическая разность	$A - B$	$\max(a - b,\, 0)$
Арифметическое произведение	$A \cdot B$	$\min(a \cdot b,\, u)$
Арифметическое деление	A / B	$\begin{cases} \lfloor a/b \rfloor, & b > 0, \\ 0, & b = 0 \end{cases}$

Разность и симметрическая разность выражены через базовые операции (объединение, пересечение, дополнение) — именно так они и реализованы в программе. Формула симметрической разности получается подстановкой: $A \triangle B = (A \cup B) \cap \overline{A \cap B}$, откуда $\min\big(\max(a,b),\, u - \min(a,b)\big)$, где $a = m_A(x)$, $b = m_B(x)$, $u = m_U(x)$. Эта запись равносильна определению $A \triangle B = (A \setminus B) \cup (B \setminus A)$, то есть выражению $\max\big(\min(a, u-b),\, \min(b, u-a)\big)$: при $a \geq b$ обе формулы дают $\min(a, u - b)$, а случай $a < b$ симметричен.

Замечание. Так как разность определена через дополнение до универсума, она отличается от арифметической разности: $A \setminus B$ уменьшает кратность не на $m_B(x)$, а ограничивает её величиной $m_U(x) - m_B(x)$. В частно-

сти, в общем случае $A \setminus A \neq \varnothing$ (кратность равна $\min(a, u-a)$). Для получения «вычитания кратностей» служит арифметическая разность $A - B$.

Все результаты операций корректны, то есть $0 \le m_C(x) \le m_U(x)$ при $A, B \subseteq U$; в частности, дополнение вычисляется без выхода в отрицательные значения.

1.3 Алгоритм генерации бинарного кода Грея

Бинарным кодом Грея разрядности n называется последовательность всех 2^n двоичных наборов длины n, в которой соседние наборы отличаются ровно в одном разряде [**?**]. Код является циклическим: последний и первый наборы также отличаются в одном разряде.

i-й код Грея вычисляется непосредственно по формуле

$$g(i) = i \oplus (i \gg 1), \qquad i = 0, 1, \ldots, 2^n - 1, \tag{3}$$

где $\oplus$ — побитовое исключающее «ИЛИ», а $\gg$ — сдвиг вправо на один разряд. Полученное число записывается в виде строки из n двоичных цифр с сохранением ведущих нулей. Построение для $n = 3$ приведено в таблице **??**.

Таблица 2 – Код Грея для $n = 3$

i	Двоичный код i	$i \gg 1$	Код Грея $g(i)$
0	000	000	000
1	001	000	001
2	010	001	011
3	011	001	010
4	100	010	110
5	101	010	111
6	110	011	101
7	111	011	100

Обоснование корректности. Пусть t — число младших единиц в двоичной записи i. Тогда $i \oplus (i+1) = 2^{t+1} - 1$. Операция сдвига перестановочна с $\oplus$, поэтому

$$g(i) \oplus g(i+1) = \big(i \oplus (i+1)\big) \oplus \big((i \oplus (i+1)) \gg 1\big) = (2^{t+1} - 1) \oplus (2^t - 1) = 2^t,$$

то есть соседние коды различаются ровно в одном разряде. Для замыкания цикла: $g(2^n - 1) = (2^n - 1) \oplus (2^{n-1} - 1) = 2^{n-1}$, а $g(0) = 0$ — наборы

различаются в одном (старшем) разряде. Отображение g взаимно однозначно на $[0; 2^n - 1]$ (исходное число восстанавливается побитовым накоплением $\oplus$ от старшего разряда), поэтому все 2^n кодов различны и в последовательности присутствуют все двоичные наборы длины n.

Каждый код формируется за $O(n)$ операций, поэтому построение всей последовательности имеет сложность $O(n \cdot 2^n)$. При $n = 0$ последовательность считается пустой, а универсум — пустым мультимножеством.

2 Особенности реализации

2.1 Структура программы

Программа написана на языке C++ (стандарт C++17) с использованием стандартной библиотеки [**?**] и разделена на модули по зонам ответственности (таблица **??**); связи между модулями показаны на рисунке **??**.

Таблица 3 – Модули программы

Файлы	Назначение
,	класс — хранение мультимножества и базовый доступ к кратностям
,	генерация кодов Грея в виде строк
,	операции над мультимножествами (теоретико-множественные и арифметические)
,	защищённый ввод чисел, ручное и автоматическое заполнение, вывод мультимножеств
	пользовательское меню, создание универсума и мультимножеств, вывод результатов операций

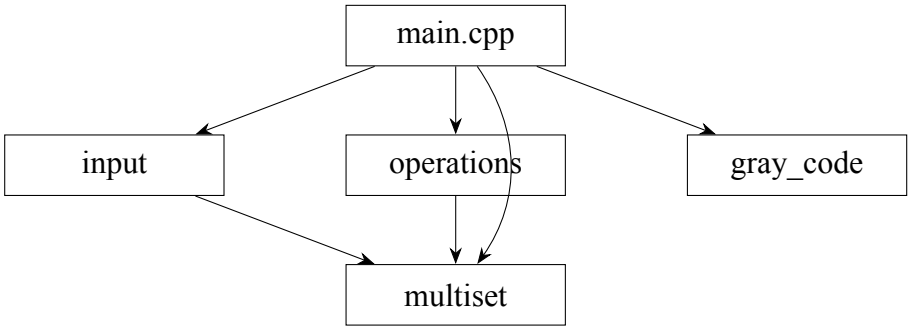


Рис. 1 – Зависимости между модулями (стрелка означает «использует»)

2.2 Структура данных: класс MultiSet

Мультимножество хранится в классе в виде ассоциативного контейнера : ключом является код Грея (строка), значением — его кратность. Контейнер хранит только элементы с положительной кратностью; метод игнорирует добавление с нулевым количеством, поэтому инвариант «в нет нулевых кратностей» выполняется всегда. Это даёт разреженное представление (пустое мультимножество не занимает памяти под элементы универсума) и упорядоченный перебор элементов, благодаря чему вывод детерминирован. Объявление класса приведено в листинге **??**.

Листинг 1 – Объявление класса MultiSet (multiset.hpp)



Методы класса и их сложность (k — число различных элементов в мультимножестве) перечислены в таблице **??**.

Таблица 4 – Методы класса MultiSet

Метод	Описание	Сложность
	удаляет все элементы	$O(k)$
	увеличивает кратность элемента на ; при ничего не делает (по умолчанию добавляется одно вхождение)	$O(\log k)$
	возвращает кратность элемента, либо 0, если элемента нет; использует , поэтому не создаёт записей в контейнере	$O(\log k)$
	мощность мультимножества — сумма всех кратностей	$O(k)$
	признак пустоты (нет ни одного элемента с кратностью > 0)	$O(1)$
	константная ссылка на внутренний контейнер для перебора элементов	$O(1)$

2.3 Генерация кода Грея

Модуль содержит две функции. Функция возвращает вектор из 2^{bits} строк: для каждого i вычисляется число $i \oplus (i \gg 1)$, которое переводится в строку функцией . Последняя строит строку ровно из символов, извлекая биты

от старшего к младшему выражением ; поэтому ведущие нули сохраняются. При возвращается пустой вектор. Реализация приведена в листинге **??**.

Листинг 2 – Функции генерации кода Грея (gray_code.cpp)

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

2.4 Ввод и вывод

Защищённое чтение числа. Функция читает строку целиком функцией , помещает её в и принимает значение только в том случае, если одновременно выполняются три условия: число успешно извлечено, после него в строке нет посторонних символов и значение лежит в допустимом диапазоне. Такая схема исключает частичное чтение, свойственное (например, строка была бы прочитана как 12). Число читается в , поэтому переполнение не приводит к ошибке (значение просто не попадает в диапазон). При некорректном вводе выводится сообщение с указанием диапазона и запрос повторяется (листинг **??**).

Листинг 3 – Функция защищённого ввода readInt (input.cpp)

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

Ручное заполнение. Функция показывает пользователю пронумерованный универсум, а затем для каждого из вхождений запрашивает *номер* элемента (а не его код), что исключает ошибки в записи кодов. Введённый номер проверяется на принадлежность диапазону $[1;\ 2^n]$; затем проверяется, что кратность элемента в формируемом мультимножестве ещё меньше его кратности в универсуме. Если лимит исчерпан, выводится сообщение, счётчик не увеличивается и ввод повторяется. Таким образом, условие $A \subseteq U$ выполняется по построению.

Автоматическое заполнение. Функция реализует случайную выборку без возвращения («алгоритм мешка»):

1) формируется вектор , в который каждый элемент универсума помещается столько раз, какова его кратность, — всего $|U|$ строк;

2) вектор перемешивается алгоритмом с генератором , инициализируемым значением ;

3) первые элементов добавляются в результат.

Элемент не может быть извлечён больше раз, чем он находится в мешке, поэтому условие $A \subseteq U$ выполняется без дополнительных проверок, а мощность результата равна . Время и память — $O(|U|)$ (листинг **??**).

Листинг 4 – Автоматическое заполнение мультимножества (input.cpp)

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Вывод. Функция печатает универсум в порядке кода Грея с нумерацией, кратностями и общей мощностью; печатает мультимножество в формате , а пустое — как $\varnothing$.

2.5 Операции над мультимножествами

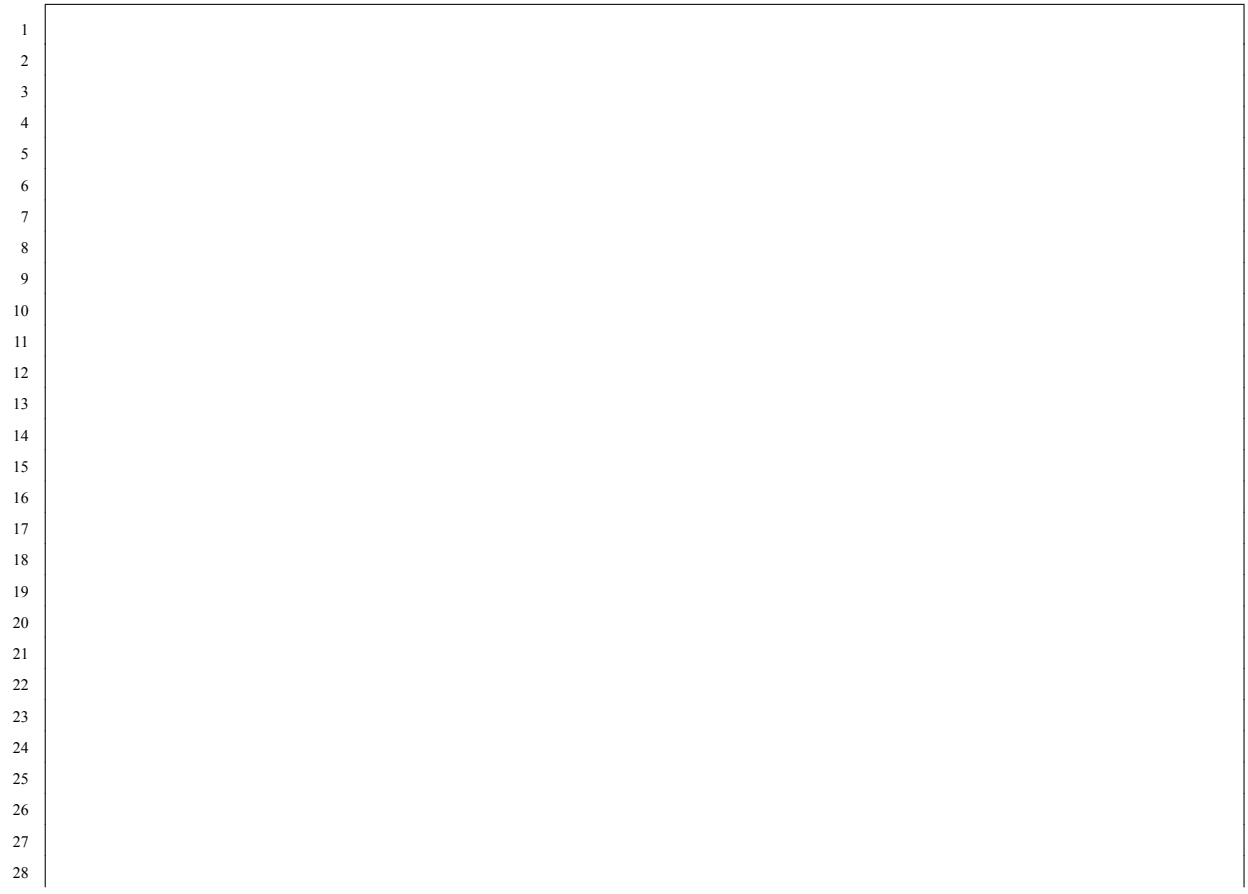
Модуль содержит по одной функции на каждую операцию задания; все функции возвращают новый объект , не изменяя операнды. Соответствие функций формулам таблицы **??** показано в таблице **??**. Для перебора достаточно одного прохода по элементам первого операнда (или обоих — при объединении), а кратность во втором операнде находится методом , поэтому сложность каждой базовой операции составляет $O(k \log k)$.

Таблица 5 – Функции модуля operations

Функция	Операция	Принцип реализации
	$A \cup B$	копия A; для элементов B кратность увеличивается до максимума
	$A \cap B$	минимум кратностей по элементам A
	$\overline{A}$	$m_U(x) - m_A(x)$ по всем элементам универсума
	$A \setminus B$	
	$A \triangle B$	
	$A + B$	сумма кратностей, ограниченная $m_U(x)$
	$A - B$	разность кратностей, только положительные результаты
	$A \cdot B$	произведение кратностей (при $m_B(x) > 0$), ограниченное $m_U(x)$
	A/B	целая часть частного при $m_B(x) > 0$; при $m_B(x) = 0$ элемент не включается

Разность и симметрическая разность выражены композицией базовых операций, поэтому код дословно повторяет математические определения (листинг **??**). Деление на нулевую кратность исключено явной проверкой, что предотвращает аварийное завершение.

Листинг 5 – Фрагмент модуля операций (operations.cpp)



```
29
30
31
32
```

2.6 Главная функция и меню

Состояние программы хранится в : универсум , вектор с кодами в порядке Грея (он задаёт нумерацию при ручном вводе) и мультимножества и . Максимальная разрядность ограничена константой , что соответствует $2^{20} = 1\,048\,576$ различным кодам. Меню содержит четыре пункта (таблица **??**).

Таблица 6 – Пункты меню

Пункт	Название	Действие
1	Создать новые мультимножества	запрос разрядности, генерация универсума, ввод мощностей и способов заполнения, создание A и B
2	Показать универсум и мультимножества	вывод U, A и B
3	Выполнить операции над мультимножествами	вывод A, B и результатов всех операций
0	Выход	завершение программы

Функция выполняет пункт 1 в следующем порядке:

1) генерирует коды Грея заданной разрядности и очищает предыдущие данные;

2) при разрядности 0 сообщает, что универсум, A и B пусты, и завершает работу (граничный случай);

3) назначает каждому коду случайную кратность из $[1; 7]$ (равномерное распределение , генератор);

4) выводит универсум и время выполнения этапов (генерация кодов, заполнение, печать), измеренное с помощью ;

5) запрашивает мощности A и B в диапазоне $[0; |U|]$ и способы их заполнения (1 — вручную, 2 — автоматически);

6) заполняет мультимножества выбранным способом и выводит их.

Функция печатает результаты операций: $A \cup B$, $A \cap B$, $A \setminus B$, $B \setminus A$, $A \triangle B$, $\overline{A}$, $\overline{B}$, а также $A + B$, $A - B$, $B - A$, $A \cdot B$, A/B и B/A. Вычитание, деление не коммутативны, поэтому выводятся оба варианта; произведение и сумма коммутативны и выводятся один раз.

2.7 Защита от некорректного ввода

Реакция программы на некорректные действия пользователя приведена в таблице **??**.

Таблица 7 – Реакция программы на некорректный ввод

Некорректный ввод	Реакция программы
нечисловая строка (), число с посторонними символами (), дробное число (), два числа (), пустая строка	сообщение «Ошибка: введите целое число от … до …», повторный запрос
число вне диапазона (разрядность > 20 или < 0, мощность $> \|U\|$, пункт меню, отличный от 0–3)	то же сообщение с указанием допустимого диапазона
число, не помещающееся в	значение отклоняется как нечисловое, программа не завершается аварийно
пробелы по краям числа ()	значение принимается как 5
номер элемента вне $[1; 2^n]$ при ручном вводе	сообщение об ошибке, повторный запрос
элемент уже использован максимальное число раз при ручном вводе	сообщение с указанием максимальной кратности, ввод повторяется, счётчик вхождений не увеличивается
способ заполнения, отличный от 1 и 2	сообщение об ошибке, повторный запрос
разрядность 0	пустые универсум, A и B; операции дают пустые результаты
пункты 2 и 3 до создания мультимножеств	выводятся пустые универсум и мультимножества, аварийного завершения нет

2.8 Оценка сложности

Генерация универсума имеет сложность $O(n \cdot 2^n)$ по времени, заполнение контейнера — $O(2^n \cdot n)$ (каждая вставка в сравнивает строки длины n), вывод — $O(2^n)$. Автоматическое заполнение требует $O(|U|)$ времени и памяти, ручное — $O(power \cdot \log 2^n)$. Операции над мультимножествами выполняются за $O(k \log k)$, где $k \leq 2^n$ — число различных элементов. Экспериментальные замеры приведены в п. **??**.

3 Результаты работы

Ниже приведены протоколы работы программы, полученные при запуске собранного приложения (сборка:). Значения кратностей универсума и результаты автоматического заполнения случайны и от запуска к запуску различаются. Введённые пользователем данные в протоколах показаны после приглашений.

Корректный ввод

3.1 Главное меню

После запуска программа выводит меню из четырёх пунктов и ожидает выбора (рисунок **??**).

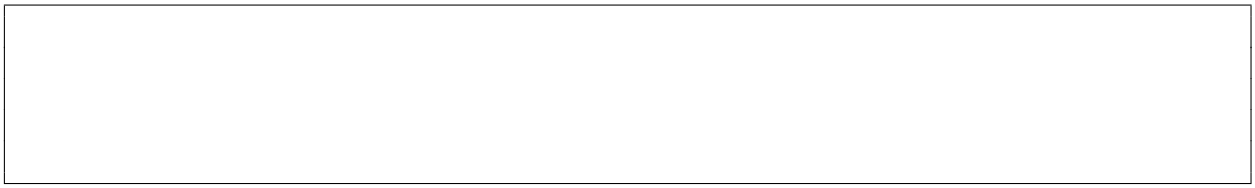


Рис. 2 – Главное меню

3.2 Создание универсума и автоматическое заполнение

Выбирается пункт 1, вводится разрядность 3 (допустимый диапазон — от 0 до 20). Программа формирует 8 кодов Грея в порядке кода Грея со случайными кратностями из $[1; 7]$ и выводит универсум с его мощностью. Затем задаются мощности $A = 5$ и $B = 4$ и автоматический способ заполнения (рисунок **??**). Пункт 2 повторно выводит универсум и оба мультимножества.

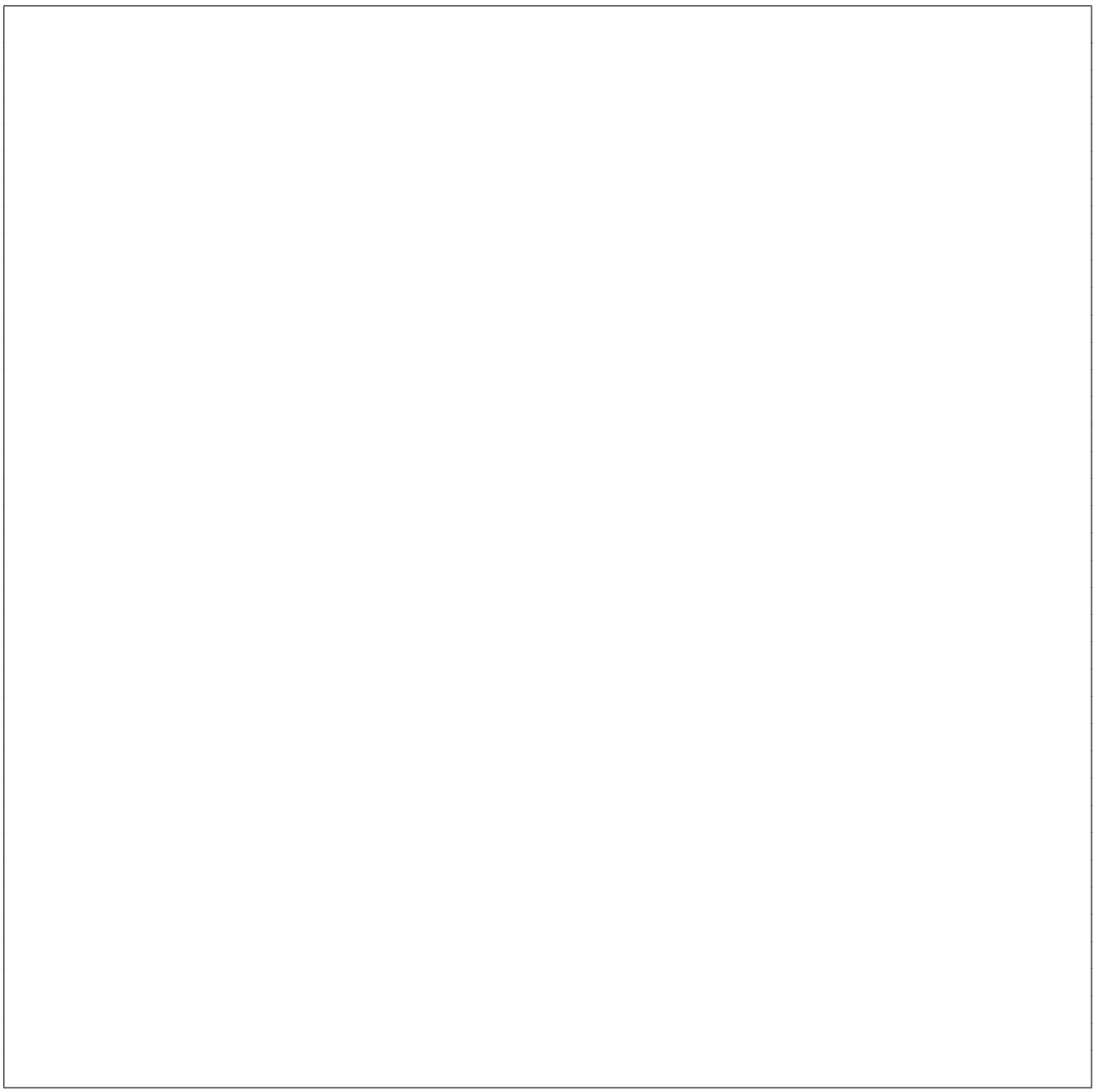


Рис. 3 – Создание универсума и автоматическое заполнение A и B

Мощности полученных мультимножеств равны заданным ($|A| = 5$, $|B| = 4$), кратности не превышают кратностей универсума.

3.3 Ручное заполнение

Выбирается ручной способ заполнения для A (мощность 8) и автоматический для B (мощность 0). В протоколе (рисунок **??**) намеренно вводятся ошибочные значения: номер элемента вне диапазона, нечисловая строка, а также повторный ввод элемента, кратность которого в универсуме уже исчерпана.



Рис. 4 – Ручное заполнение мультимножества A с ошибками ввода

Мультимножество A сформировано в точном соответствии с допустимыми введёнными данными: при превышении лимита (— 3 вхождения, — 4) программа сообщила об ошибке и запросила другой элемент, а мощность $|A| = 8$ равна заданной.

3.4 Выполнение операций

Сначала выбирается пункт 1: вводится разрядность 3, мощности $A = 8$ и $B = 6$ и автоматический способ заполнения обоих мультимножеств (рисунок **??**).



Рис. 5 – Универсум и мультимножества A и B для проверки операций

Затем выбирается пункт 3. Программа выводит результаты всех операций над A и B (рисунок **??**).

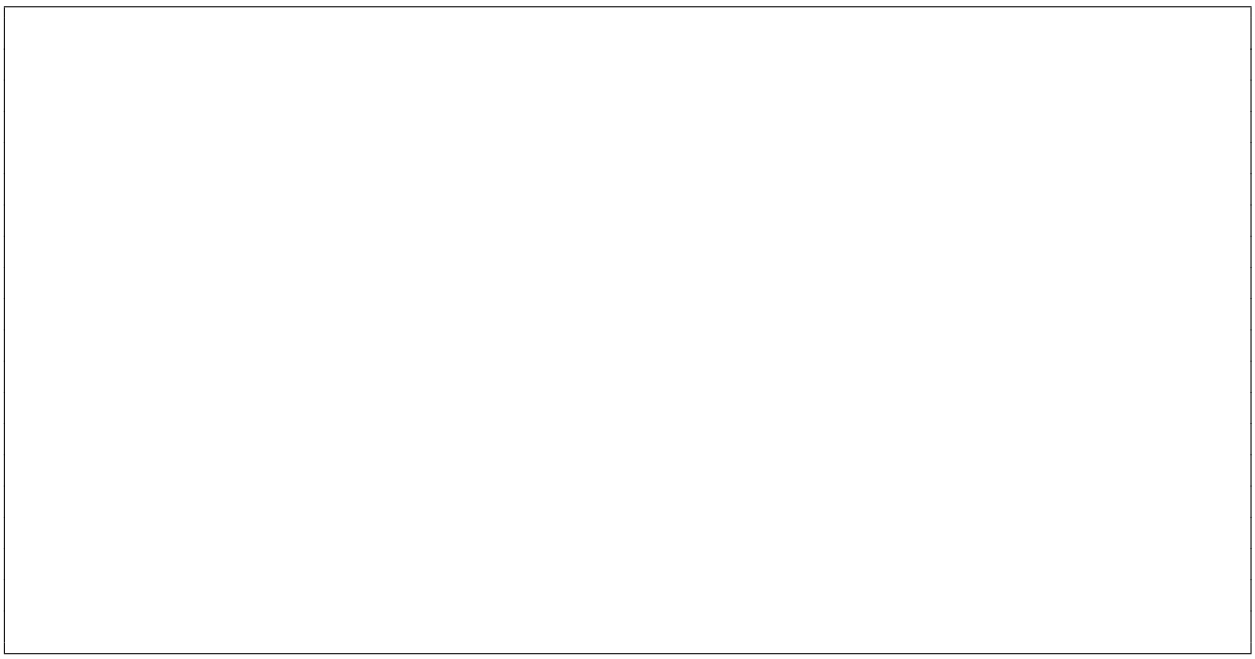


Рис. 6 – Результаты операций над A и B

Для проверки правильности вычислим результаты вручную по формулам таблицы **??** для трёх элементов (таблица **??**). Значения a, b, u взяты из протоколов на рисунках **??** и **??**.

Таблица 8 – Проверка результатов операций вручную

	$x = 010$	$x = 101$	$x = 111$
$(a;\, b;\, u)$	$(2;\, 1;\, 7)$	$(1;\, 2;\, 6)$	$(2;\, 0;\, 7)$
$A \cup B$	2	2	2
$A \cap B$	1	1	0
$A \setminus B$	$\min(2, 6) = 2$	$\min(1, 4) = 1$	$\min(2, 7) = 2$
$B \setminus A$	$\min(1, 5) = 1$	$\min(2, 5) = 2$	$\min(0, 5) = 0$
$A \triangle B$	$\min(2, 6) = 2$	$\min(2, 5) = 2$	$\min(2, 7) = 2$
$\overline{A}$	$7 - 2 = 5$	$6 - 1 = 5$	$7 - 2 = 5$
$\overline{B}$	$7 - 1 = 6$	$6 - 2 = 4$	$7 - 0 = 7$
$A + B$	$\min(3, 7) = 3$	$\min(3, 6) = 3$	2
$A - B$	1	0	2
$B - A$	0	1	0
$A \cdot B$	$\min(2, 7) = 2$	$\min(2, 6) = 2$	0
A/B	$\lfloor 2/1 \rfloor = 2$	$\lfloor 1/2 \rfloor = 0$	0
B/A	$\lfloor 1/2 \rfloor = 0$	$\lfloor 2/1 \rfloor = 2$	0

Значения совпадают с выводом программы: например, в рисунке **??** для элемента присутствуют $\cup$ — 2, $\cap$ — 1, $+$ — 3, $-$ — 1, а для — равно 2 и равно 0 (элемент в результат не входит).

3.5 Граничные случаи

При разрядности 0 универсум и оба мультимножества пусты; операции над пустыми мультимножествами также дают пустые результаты, программа не завершается аварийно (рисунок **??**).

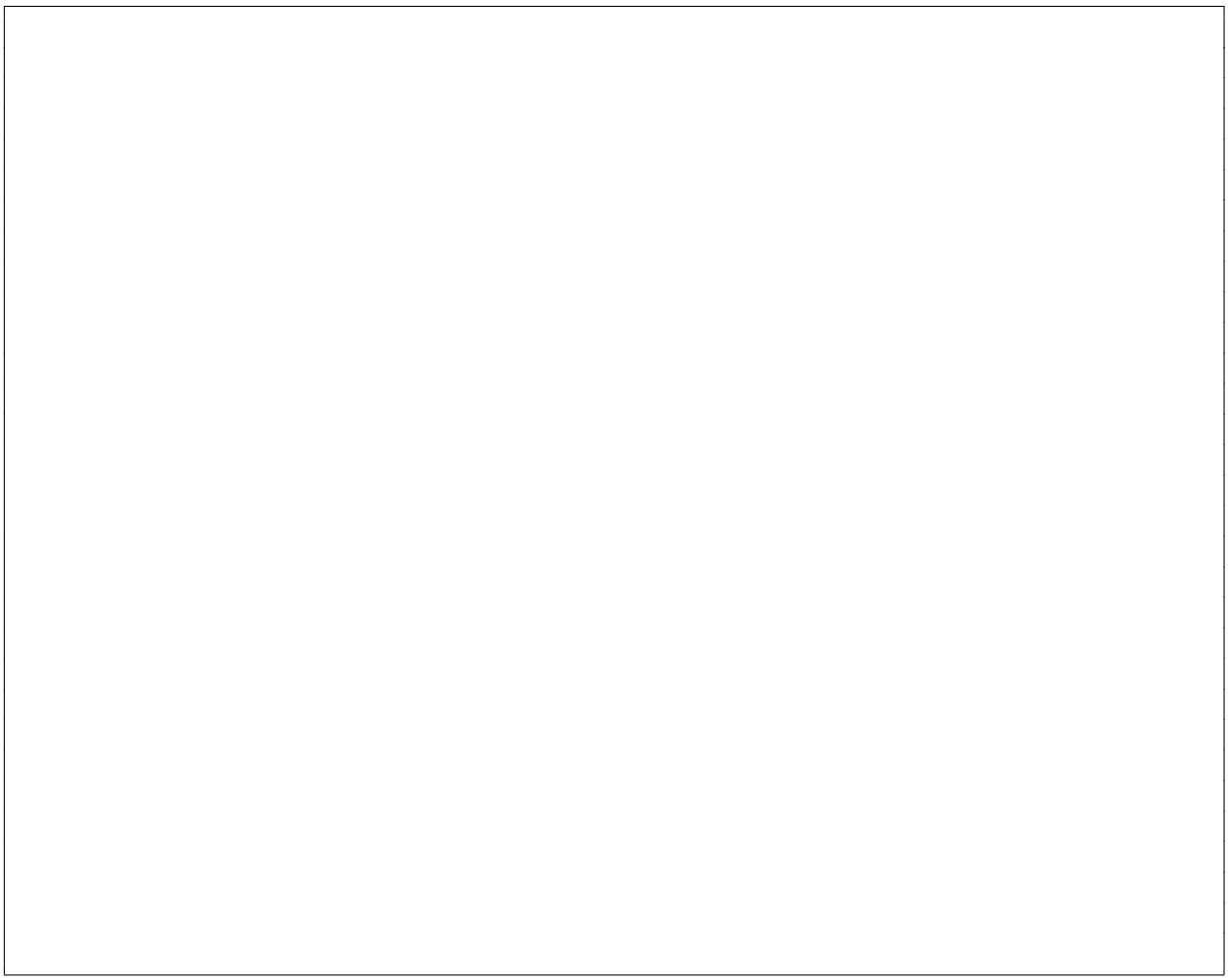


Рис. 7 – Разрядность 0: пустые мультимножества и результаты операций

Некорректный ввод

3.6 Некорректный пункт меню

В меню вводятся , , и пустая строка. Каждый ввод отклоняется с сообщением о допустимом диапазоне, затем вводится — выход (рисунок **??**).

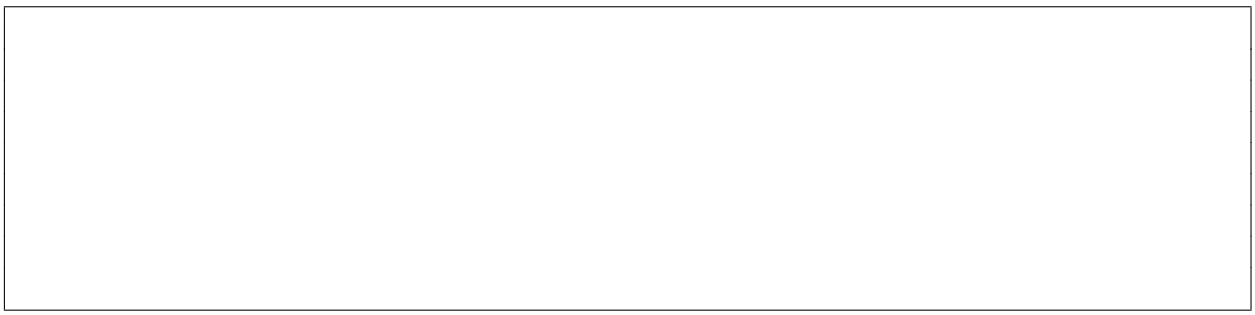


Рис. 8 – Некорректный ввод в меню

3.7 Некорректная разрядность

При создании универсума вводятся , , и . Все значения отклоняются, после чего принимается корректное значение 2 (рисунок **??**).

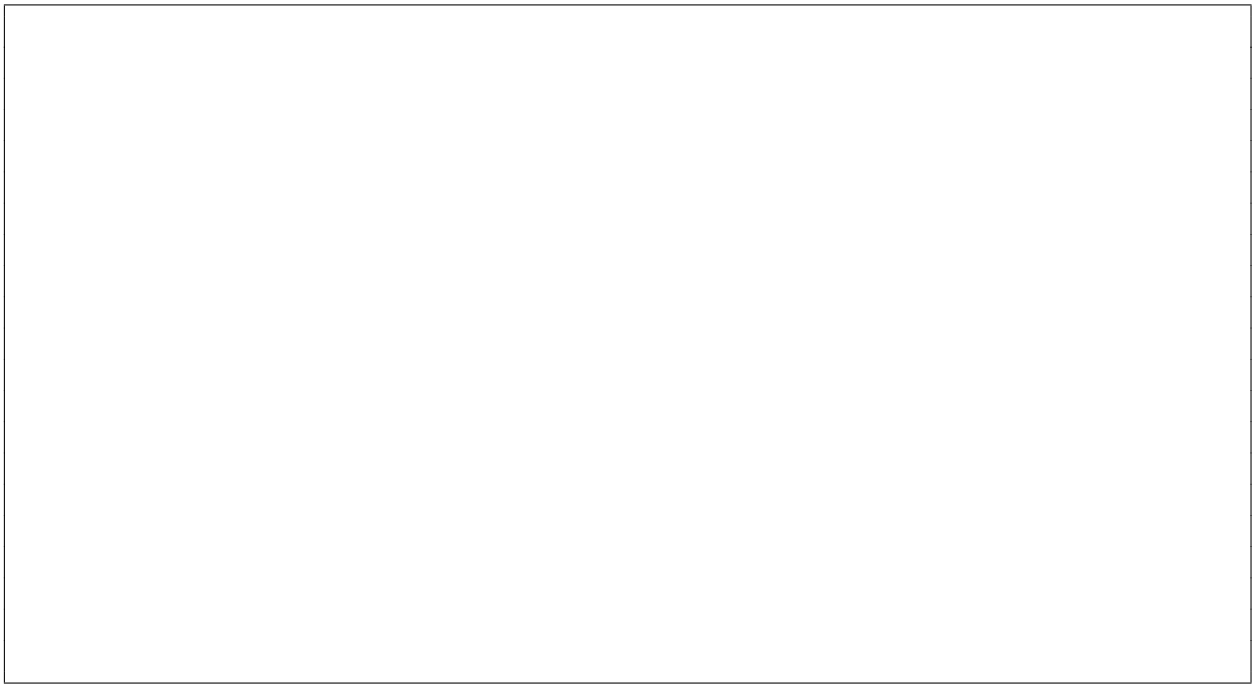


Рис. 9 – Некорректная разрядность кода Грея

3.8 Мощность больше мощности универсума и неверный способ заполнения

При вводе мощности 99 при $|U| = 24$ и нечисловой строки ввод отклоняется. Для способа заполнения введено недопустимое значение 7 (рисунок **??**).

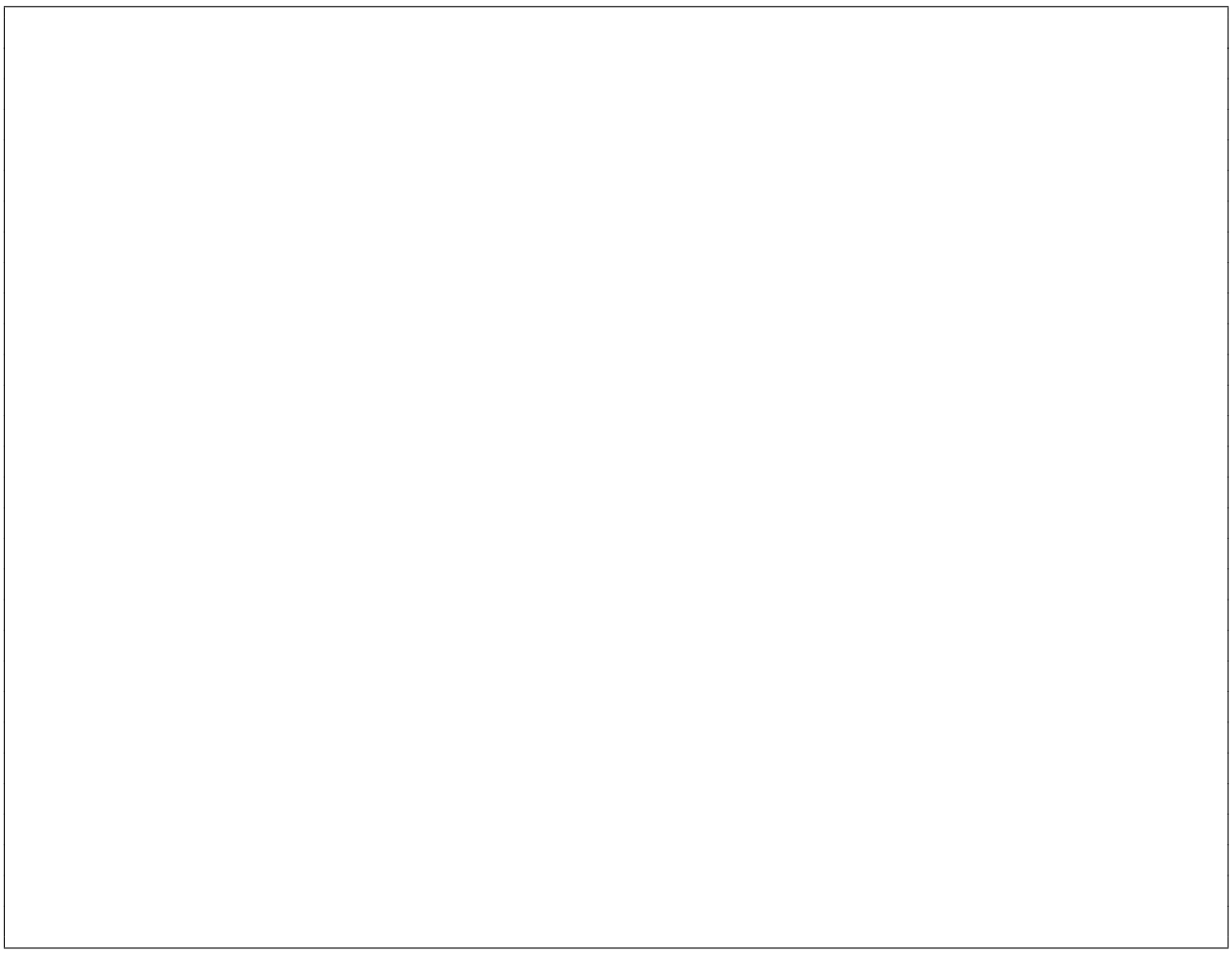


Рис. 10 – Мощность больше $|U|$ и неверный способ заполнения

3.9 Замеры времени

Для оценки масштабируемости программа выводит время генерации кодов, заполнения универсума и печати. В таблице **??** приведены результаты для разных разрядностей. Замеры ориентировочные: получены при сборке , вывод перенаправлен в канал, кратности универсума случайны.

Таблица 9 – Время выполнения этапов создания универсума

n	2^n	$\lvert U\rvert$	Генерация кодов, мс	Заполнение, мс	Печать, мс
10	1 024	4 068	< 1	< 1	< 1
14	16 384	65 991	3	6	9
16	65 536	261 463	4	29	36
18	262 144	1 046 518	26	84	112
20	1 048 576	4 193 325	69	381	491

Время растёт примерно пропорционально 2^n, что согласуется с оценкой

$O(n \cdot 2^n)$; даже при максимальной разрядности $n = 20$ создание универсума занимает менее секунды.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована программа на C++, которая:

– генерирует бинарный код Грея разрядности $n \in [0; 20]$ по формуле $g(i) = i \oplus (i \gg 1)$;

– формирует универсум из 2^n кодов Грея со случайными кратностями из диапазона $[1; 7]$;

– формирует мультимножества A и B заданной мощности вручную или автоматически;

– вычисляет и выводит объединение, пересечение, разность (оба варианта), симметрическую разность, дополнение (оба варианта), арифметические сумму, разность, произведение и деление;

– защищена от некорректного ввода и управляется через меню.

Достоинства реализации:

– чёткое разделение ответственности между модулями (хранение, генерация, операции, ввод-вывод, интерфейс);

– разреженное хранение мультимножества в : в памяти находятся только элементы с положительной кратностью, вывод упорядочен и детерминирован;

– разность и симметрическая разность выражены через базовые операции, поэтому код совпадает с математическими определениями;

– условие $A \subseteq U$ гарантируется при обоих способах заполнения; автоматическое заполнение не требует дополнительных проверок;

– строка ввода читается и проверяется целиком, что исключает частичное чтение значений;

– встроенные замеры времени позволяют оценить производительность при больших разрядностях.

Недостатки реализации:

– автоматическое заполнение требует $O(|U|)$ дополнительной памяти на «мешок» из строк (при $n = 20$ — порядка нескольких миллионов строк);

– при больших разрядностях печать всего универсума (2^n строк) занимает большую часть времени и засоряет консоль;

– значения кратностей хранятся в , а строки кодов дублируются в ключах , что увеличивает расход памяти по сравнению с вектором кратностей, индексированным номером кода;

– не предусмотрена обработка завершения потока ввода (Ctrl+D / EOF);

– результаты операций нельзя сохранить в файл или использовать как операнды следующей операции.

Масштабирование. На основе реализованных функций программу можно дополнить с минимальными изменениями кода:

– пунктом меню, сохраняющим результат выбранной операции в A или B (присваиванием возвращённого функцией объекта), что позволит строить цепочки вычислений;

– проверкой отношений между мультимножествами: включением $A \subseteq B$ ($m_A(x) \leq m_B(x)$ для всех x) и равенством $A = B$ — одной функцией, возвращающей , за один проход по элементам;

– операциями над тремя и более мультимножествами, например $A \cap B \cap C$ или $(A \cup B) \setminus C$, так как функции операций возвращают новый объект и применяются последовательно;

– арифметической степенью A^k с кратностями $\min(m_A(x)^k,\ m_U(x))$ и умножением на число $k \cdot A$ — повторным применением произведения и суммы;

– вводом и выводом мультимножеств в файл: достаточно направить в и реализовать чтение пар «код; кратность» с той же проверкой диапазонов, что и в ;

– заменой строкового ключа на целочисленный индекс кода (или битовое представление), что уменьшит расход памяти и ускорит операции.

СПИСОК ЛИТЕРАТУРЫ

Список литературы

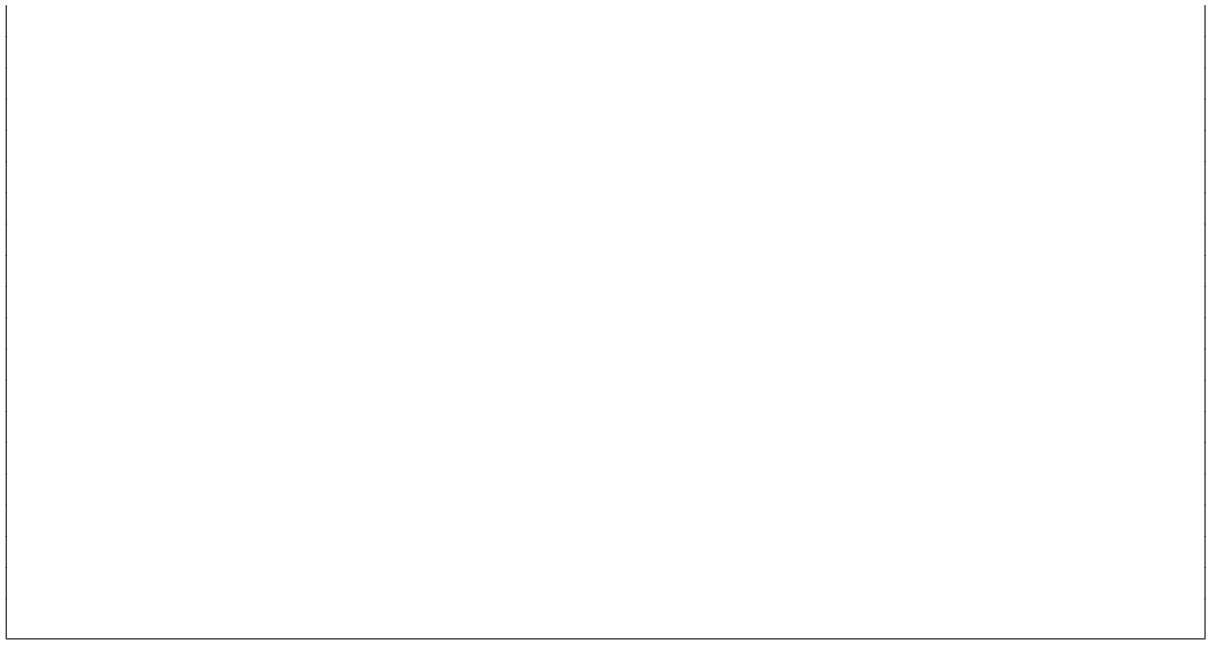
[1] Новиков Ф. А. Дискретная математика для программистов: учебник для вузов. — 3-е изд. — СПб.: Питер, 2009. — 384 с.

[2] Knuth D. E. The Art of Computer Programming. Vol. 4A: Combinatorial Algorithms, Part 1. — Upper Saddle River: Addison-Wesley, 2011. — 883 p.

[3] cppreference.com: справочник по языку C++ и стандартной библиотеке [Электронный ресурс]. — URL: (дата обращения: 24.10.2026).

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

Листинг А.1 – main.cpp



Листинг А.2 – multiset.hpp



Листинг А.3 – multiset.cpp

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
```

Листинг А.4 – input.hpp

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
```

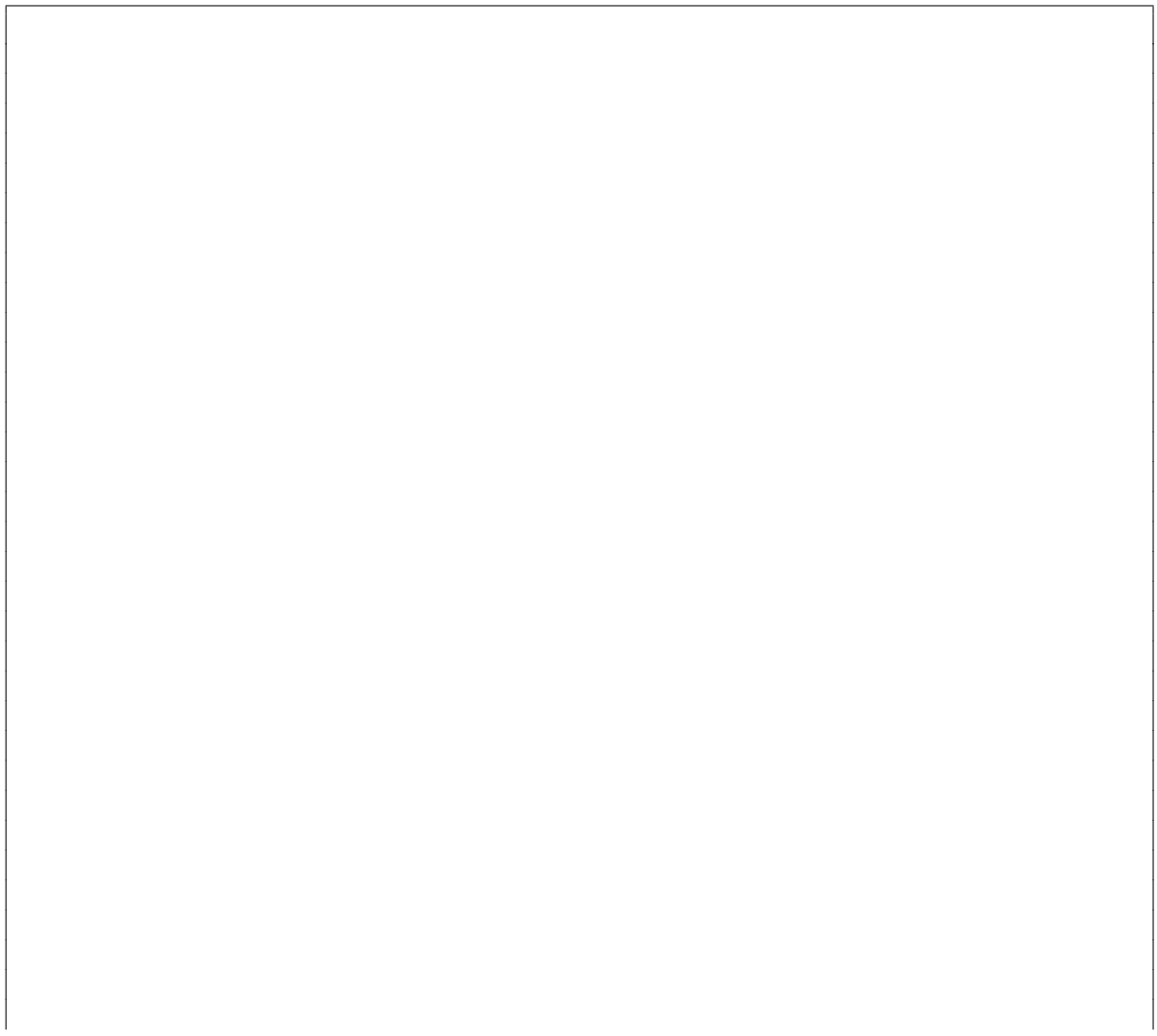
Листинг А.5 – input.cpp

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
```

Листинг А.6 – operations.hpp



Листинг А.7 – operations.cpp



```
97
98
99
```

Листинг А.8 – gray_code.hpp

```
1
2
3
4
5
6
7
8
```

Листинг А.9 – gray_code.cpp

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
```